

SpendWise: Production Readiness & QA Architecture Report

Target Application: SpendWise PWA (Next.js 14, Zustand, Firebase/Firestore, Framer Motion, Recharts) | **Auditor Role:** Lead QA Architect & Principal UX Designer | **Theme System:** Sahara Editorial Layout

1. Executive Summary & Audit Scorecard

SpendWise is an exceptionally well-conceived Progressive Web Application (PWA) tailored for college students, combining high-end editorial typography (*EB Garamond* & *Manrope*) with a "Thick-Client" architecture. While the foundational visual identity and month-based Firestore sharding demonstrate strong systems thinking, rigorous QA testing exposed critical calculation crashes, listener memory leaks, and UI density flaws that prevent production readiness.

This master blueprint aggregates **every identified flaw, UX inconsistency, architectural hazard, and proposed code-level remediation** across the entire audit, providing complete, ready-to-implement TypeScript solutions.

Visual Identity (Sahara Theme)

9.0 / 10

Distinctive warm peach/brown palette and serif headers far superior to generic Material/Bootstrap student apps. Requires tabular figure discipline for ledgers.

QA Rigor & Math Engine

5.0 / 10

Critical bugs identified: `62800% used` division-by-zero crashes, broken spacing in Rupee strings, and unconstrained desktop form stretching.

Frontend State Architecture

8.5 / 10

Clean Zustand modular slices and month-based Firestore sharding. Needs atomic rollback guards and listener `AbortController` refactoring.

Data Density & Scannability

6.5 / 10

Square transaction cards waste screen real estate. Requires high-density ledger rows, daily burn-rate widgets, and dual-progress analytics bars.

2. Critical QA Bugs & Root-Cause Remediation

The QA test suite identified three immediate, user-facing bugs that corrupt visual metrics and degrade layout integrity. Below is the technical breakdown and mandatory code remediation for each.

BUG #01 — Analytics Page: The "62800% Used" Math Explosion

Observed Behavior: The top-right badge on the Monthly Trend card displays `62800% used` when Total Spend is ₹628 and Budget is ₹8,000. Additionally, when a user has not set a monthly budget (`budgetAmount === 0`), the UI evaluates to `Infinity` or `NaN`.

Root Cause: Erroneous percentage math multiplying raw spend by 100 without dividing by the limit, or multiplying an already percentage-converted decimal by 100 again.

Remediation — Clamped Percentage Utility: Replace inline calculations with a safe bounding utility that clamps badges between `0.0%` and `999.9%`:

```
// src/lib/finance-math.ts – Clamped Percentage Calculation
export function calculateSpendPercentage(spent: number, limit: number): number {
  if (!limit || limit <= 0) return 0; // Prevent Division-by-Zero / Infinity
  const rawPercentage = (spent / limit) * 100;
  return Number(Math.min(rawPercentage, 999.9).toFixed(1));
}
```

BUG #02 — Analytics Page: "Ghost" Carousel Dots on Desktop

Observed Behavior: On wide desktop displays, all three analytics cards (`TOTAL SPENT`, `CATEGORY SPLIT`, `MONTHLY TREND`) fit inside the viewport simultaneously, yet the `...` pagination indicator remains rendered below them.

Root Cause: The Framer Motion carousel indicator does not evaluate viewport capacity against item count.

Remediation — Conditional Rendering Guard: Wrap the pagination dots container in an explicit count evaluation:

```
{itemsCount > visibleItemsCount && (
  <div className="flex justify-center items-center gap-2 mt-4">
    {/* Carousel dots render ONLY when overflow exists */}
  </div>
)}
```

BUG #03 — Expenses Page: Broken String Concatenation & Currency Spacing

Observed Behavior: Numerical labels render with floating whitespace (`₹628 .00` and `₹ 266 .00`) and malformed percentage strings (`+ 0 % from last month`).

Root Cause: Raw string concatenation instead of native Internationalization (i18n) number formatting.

Remediation — Centralized Indian Rupee Formatter: Utilize native JavaScript `Intl.NumberFormat` configured for Indian Rupees (INR) across all components:

```
// src/lib/utils.ts – Standardized INR Currency Formatter
export const formatCurrency = (amount: number): string => {
  return new Intl.NumberFormat('en-IN', {
    style: 'currency',
    currency: 'INR',
    maximumFractionDigits: 0, // Removes awkward .00 decimal clutter for students
  }).format(amount);
};
```

3. Page-by-Page UI/UX & Information Architecture Breakdown

PAGE / MODULE	QA & DESIGN CRITIQUE	ENGINEERED SOLUTION & UI UPGRADE
Dashboard	Inconsistent Rupee symbols (Current Balance lacks ₹). "Recent Expenses" renders as large, inefficient square cards (Food ₹60 , Food ₹20) that destroy vertical scannability.	Standardize all figures via <code>formatCurrency()</code> . Replace square cards with a high-density, horizontal ledger list (<code>TransactionRow</code>) to maximize data density.
Expenses Page	Top-right "Inspirational Quote" wallet photo card wastes prime real estate. Redundant minus signs (- ₹60.00) add visual clutter in an expense-only ledger.	Replace quote card with a Daily Safe-Spend Odometer . Remove negative prefix signs unless a dual-entry income/expense toggle is introduced.
Analytics Page	Verbose paragraphs under Category Breakdown ("You've spent ₹266.00 on Travel so far..."). Users do not read prose in analytical dashboards.	Replace prose with structured Dual-Progress Bars : Bar 1 showing % of total monthly spend, Bar 2 showing % of category limit burned.
Add Transaction (Bottom Sheet)	Font mismatch: Large amount input 0 renders in serif display font (EB Garamond), causing numbers to shift width. Right-edge category labels clip (Book . . .).	Force monospaced sans-serif (<code>tabular-nums font-manrope</code>) for numerical input fields. Ensure horizontal scroll container has proper right padding.
Settings Pages	Input fields stretch to 100% viewport width on desktop. No option to hide unused default Indian student preset categories (e.g., Snacks & Chai).	Constrain desktop form widths using <code>max-w-xl</code> . Provide a toggle in Category Settings to disable unused presets from the logging drawer.

High-Density Ledger Component (`TransactionRow.tsx`)

To solve the Dashboard data-density problem, replace square cards with this production-ready scannable list row:

```

// src/components/ledger/TransactionRow.tsx
import React from 'react';
import { CategoryIcon } from '@components/ui/CategoryIcon';
import { formatCurrency } from '@lib/utils';

interface TransactionRowProps {
  title: string;
  categoryName: string;
  amount: number;
  formattedTime: string;
  categoryColor: string;
}

export const TransactionRow: React.FC<TransactionRowProps> = ({
  title, categoryName, amount, formattedTime, categoryColor,
}) => {
  return (
    <div className="flex items-center justify-between py-3 px-4 rounded-xl bg-theme-surface
border border-theme-primary/5 hover:bg-theme-base/50 transition-colors">
      <div className="flex items-center gap-3.5 min-w-0">
        <div
          className="w-10 h-10 rounded-full flex items-center justify-center flex-shrink-0"
          style={{ backgroundColor: `${categoryColor}15`, color: categoryColor }}
        >
          <CategoryIcon name={categoryName} className="w-5 h-5" />
        </div>
        <div className="min-w-0 flex-1">
          <p className="font-manrope font-medium text-sm text-theme-primary truncate">{title}</p>
          <p className="font-manrope text-xs text-theme-secondary">{categoryName} •
{formattedTime}</p>
        </div>
        <div className="text-right flex-shrink-0 pl-4">
          {/* Tabular numbers ensure Rupee digits never shift column alignment */}
          <span className="font-manrope font-semibold text-base text-theme-primary tabular-nums
tracking-tight">
            {formatCurrency(amount)}
          </span>
        </div>
      </div>
    </div>
  );
};

```

4. Architecture & Firebase Edge-Case Remediation

SpendWise relies on a "Thick-Client" pattern where local Zustand state synchronizes directly with Firestore. The audit uncovered critical race conditions during pagination and lack of rollback guards during network failures.

4.1 Eliminating Listener Race Conditions in `useExpensesListener`

When a user rapidly clicks "Load Older Transactions", pushing months like `2026-07` and `2026-06` into Zustand's `loadedMonths` array, multiple async listeners overlap, creating memory leaks and query thrashing.

```

// src/hooks/useExpensesListener.ts – Production AbortController Refactor
import { useEffect, useRef } from 'react';
import { collection, query, where, onSnapshot, Unsubscribe } from 'firebase/firestore';
import { db } from '@lib/firebase';
import { useStore } from '@store';
import { ExpenseDocument } from '@types';

export function useExpensesListener() {
  const { householdId, loadedMonths, setExpenses, setIsExpensesLoading } = useStore((s) => ({
    householdId: s.householdId,
    loadedMonths: s.loadedMonths,
    setExpenses: s.setExpenses,
    setIsExpensesLoading: s.setIsExpensesLoading,
  }));

  const unsubscribeRef = useRef<Unsubscribe | null>(null);

  useEffect(() => {
    if (!householdId || loadedMonths.length === 0) return;
    setIsExpensesLoading(true);

    // 1. Kill orphaned listeners before spawning a new connection
    if (unsubscribeRef.current) {
      unsubscribeRef.current();
      unsubscribeRef.current = null;
    }

    // 2. Sort & deduplicate loadedMonths array to guarantee query stability
    const uniqueMonths = Array.from(new Set(loadedMonths)).sort();

    const q = query(
      collection(db, `households/${householdId}/expenses`),
      where('monthKey', 'in', uniqueMonths)
    );

    unsubscribeRef.current = onSnapshot(q,
      (snapshot) => {
        const parsedExpenses: ExpenseDocument[] = snapshot.docs.map((doc) => {
          const data = doc.data();
          return {
            id: doc.id,
            amount: Number(data.amount) || 0,
            categoryId: data.categoryId || 'misc',
            note: data.note || '',
            date: data.date?.toDate() || new Date(),
            createdAt: data.createdAt?.toDate() || new Date(),
          };
        });
        setExpenses(parsedExpenses);
        setIsExpensesLoading(false);
      },
      (error) => {
        console.error('[Firestore Sync Error]:', error);
        setIsExpensesLoading(false);
      }
    );

    return () => {
      if (unsubscribeRef.current) {
        unsubscribeRef.current();
        unsubscribeRef.current = null;
      }
    };
  });
}

```

```
    }, [householdId, JSON.stringify(loadedMonths), setExpenses, setIsExpensesLoading]);  
  }  
}
```

4.2 Atomic Rollback Pattern for Optimistic UI

To prevent state corruption when logging transactions offline or on poor campus Wi-Fi, optimistic UI updates must snapshot previous state and execute atomic rollbacks if Firestore promises reject:

```
// In src/store/expenses.slice.ts – Atomic Rollback Guard  
addExpenseOptimistic: async (newExpenseData) => {  
  const previousExpenses = get().expenses;  
  const tempId = `temp-${Date.now()}`;  
  const tempExpense: ExpenseDocument = {  
    id: tempId,  
    ...newExpenseData,  
    createdAt: new Date(),  
  };  
  
  // 1. Optimistically mutate local Zustand state immediately  
  set((state) => ({ expenses: [tempExpense, ...state.expenses] }));  
  
  try {  
    const householdId = get().householdId;  
    const monthKey = format(newExpenseData.date, 'yyyy-MM');  
  
    await addDoc(collection(db, `households/${householdId}/expenses`), {  
      ...newExpenseData,  
      monthKey,  
      date: Timestamp.fromDate(newExpenseData.date),  
      createdAt: serverTimestamp(),  
    });  
  } catch (error) {  
    // 2. Rollback to exact snapshot on network/database failure  
    set({ expenses: previousExpenses });  
    throw error;  
  }  
}
```

4.3 Zero-Backend Security: Production Firestore Rules

In a backendless application, Firestore Security Rules act as the sole API firewall. These rules enforce household ownership, authentication, and strict schema validation:

```
// firestore.rules – Production Rules Engine
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAuthenticated() {
      return request.auth != null;
    }
    function isHouseholdMember(householdId) {
      return isAuthenticated() &&
        request.auth.uid in get(/databases/{database}/documents/households/{householdId}).data.members;
    }

    match /households/{householdId} {
      allow read, update: if isHouseholdMember(householdId);
      allow create: if isAuthenticated();

      match /expenses/{expenseId} {
        allow read, delete: if isHouseholdMember(householdId);
        // Prevent negative number manipulation & validate data types
        allow create, update: if isHouseholdMember(householdId)
          && request.resource.data.amount is number
          && request.resource.data.amount > 0
          && request.resource.data.categoryId is string
          && request.resource.data.date is timestamp;
      }

      match /budget {
        allow read: if isHouseholdMember(householdId);
        allow write: if isHouseholdMember(householdId)
          && request.resource.data.amount is number
          && request.resource.data.amount >= 0;
      }
    }
  }
}
```

5. Financial Mathematical Engine & Algorithms

To elevate SpendWise beyond simple sum arithmetic, we introduce a formal mathematical engine that provides students with actionable daily burn-rate metrics and end-of-month (EOM) spending projections.

$$S_{daily} = (B_{total} - \Sigma E_i) \div (D_{total} - D_{current} + 1)$$

Where S_{daily} is the **Safe Daily Spend** allowance, B_{total} is the total monthly budget, ΣE_i is cumulative month-to-date spending, D_{total} is total days in the active month, and $D_{current}$ is the current calendar day integer.

For End-of-Month (EOM) forecasting, we calculate a **Weighted Daily Burn Rate** ($\bar{E}_{daily}$) and project final spend ($E_{projected}$):

$$E_{projected} = E_{spent} + (D_{rem} - 1) \times (E_{spent} \div D_{current})$$

Complete Mathematical Engine (`src/lib/finance-math.ts`)

```
// src/lib/finance-math.ts – Production Financial Calculation Engine
import { getDaysInMonth, getDate } from 'date-fns';

export interface FinancialSummary {
  totalSpent: number;
  remainingBudget: number;
  percentageUsed: number;
  safeDailySpend: number;
  projectedMonthlyTotal: number;
  isOverBudget: boolean;
}

export function calculateFinancialSummary(
  expenses: { amount: number; date: Date }[],
  monthlyBudget: number,
  currentDate: Date = new Date()
): FinancialSummary {
  // 1. Cumulative Monthly Spend
  const totalSpent = expenses.reduce((sum, item) => sum + item.amount, 0);

  // 2. Safe Bounds & Zero-Budget Guard
  const safeBudget = Math.max(monthlyBudget, 0);
  const remainingBudget = safeBudget - totalSpent;
  const percentageUsed = safeBudget > 0
    ? Math.min(Number(((totalSpent / safeBudget) * 100).toFixed(1)), 999.9)
    : 0;

  // 3. Calendar Math
  const totalDaysInMonth = getDaysInMonth(currentDate);
  const currentDay = getDate(currentDate);
  const daysRemaining = Math.max(totalDaysInMonth - currentDay + 1, 1);

  // 4. Safe Daily Spend Odometer
  const safeDailySpend = remainingBudget > 0
    ? Math.floor(remainingBudget / daysRemaining)
    : 0;

  // 5. Projected EOM Spend Forecast
  const averageDailyBurn = currentDay > 0 ? totalSpent / currentDay : 0;
  const projectedMonthlyTotal = Math.round(
    totalSpent + (daysRemaining - 1) * averageDailyBurn
  );

  return {
    totalSpent,
    remainingBudget,
    percentageUsed,
    safeDailySpend,
    projectedMonthlyTotal,
    isOverBudget: totalSpent > safeBudget,
  };
}
```

6. PWA Resilience, Mobile Haptics & Performance

SpendWise is designed as an offline-capable Progressive Web App. To achieve native mobile app ergonomics, we implement Web Haptics, safe-area padding for iOS devices, and dynamic JavaScript bundle splitting.

6.1 iOS Safe-Area Ergo-Padding

When installed to an iPhone Home Screen, bottom sheets and action bars clip under the Face ID gesture bar. All fixed bottom containers must apply Tailwind safe-area utility classes:

```
<div className="fixed bottom-0 w-full pb-safe bg-theme-surface">
  {/* Ensures bottom '+' button never
  collides with iOS home indicator */}
</div>
```

6.2 Dynamic Bundle Splitting

Recharts and Framer Motion are heavy libraries. Prevent them from blocking the initial PWA load by lazy-loading analytics components via `next/dynamic`:

```
const SpendAreaChart = dynamic(
  () => import('./SpendAreaChart').then((m)
=> m.SpendAreaChart),
  { ssr: false, loading: () =>
<ChartSkeleton /> }
);
```

6.3 Tactile Haptic Engine (`src/lib/haptics.ts`)

Because the custom numeric keypad bypasses native device keyboards, we trigger tactile micro-vibrations via the Web Vibration API:

```
// src/lib/haptics.ts – Mobile Tactile Feedback Engine
export const triggerHaptic = (type: 'light' | 'medium' | 'heavy' | 'warning' = 'light') => {
  if (typeof window === 'undefined' || !('vibrate' in navigator)) return;
  try {
    switch (type) {
      case 'light': navigator.vibrate(8); break; // Keypad digit tap
      case 'medium': navigator.vibrate(15); break; // Category chip select
      case 'heavy': navigator.vibrate(30); break; // Expense log success
      case 'warning': navigator.vibrate([25, 50, 25]); break; // Budget >85% alert
    }
  } catch (e) {
    /* Gracefully ignore unsupported desktop browsers */
  }
};
```

7. Verification & Unit Testing Suite

To validate the financial math engine against division-by-zero crashes and over-budget scenarios, execute this automated Vitest test suite (`src/\_\_tests\_\_/finance-math.test.ts`):

```
// src/__tests__/finance-math.test.ts - Vitest QA Suite
import { describe, it, expect } from 'vitest';
import { calculateFinancialSummary } from '@lib/finance-math';

describe('calculateFinancialSummary - QA Edge Cases', () => {
  const mockDate = new Date('2026-08-05T12:00:00Z'); // August 5th (31 total days, 27
  remaining)

  it('calculates spend percentage and safe daily spend accurately', () => {
    const expenses = [
      { amount: 500, date: new Date('2026-08-01') },
      { amount: 128, date: new Date('2026-08-02') },
    ];
    const result = calculateFinancialSummary(expenses, 8000, mockDate);

    expect(result.totalSpent).toBe(628);
    expect(result.remainingBudget).toBe(7372);
    expect(result.percentageUsed).toBe(7.9); // Clamped (628 / 8000 = 7.85%)
    expect(result.safeDailySpend).toBe(273); // Math.floor(7372 / 27 days left)
    expect(result.isOverBudget).toBe(false);
  });

  it('prevents division-by-zero Infinity and 62800% bugs when budget is 0', () => {
    const expenses = [{ amount: 628, date: new Date('2026-08-01') }];
    const result = calculateFinancialSummary(expenses, 0, mockDate);

    expect(result.percentageUsed).toBe(0); // Gracefully clamped
    expect(result.safeDailySpend).toBe(0);
    expect(result.isOverBudget).toBe(true);
  });

  it('handles over-budget scenarios without throwing negative daily spend', () => {
    const expenses = [{ amount: 9500, date: new Date('2026-08-04') }];
    const result = calculateFinancialSummary(expenses, 8000, mockDate);

    expect(result.remainingBudget).toBe(-1500);
    expect(result.safeDailySpend).toBe(0); // Cannot recommend negative daily allowance
    expect(result.isOverBudget).toBe(true);
  });
});
```

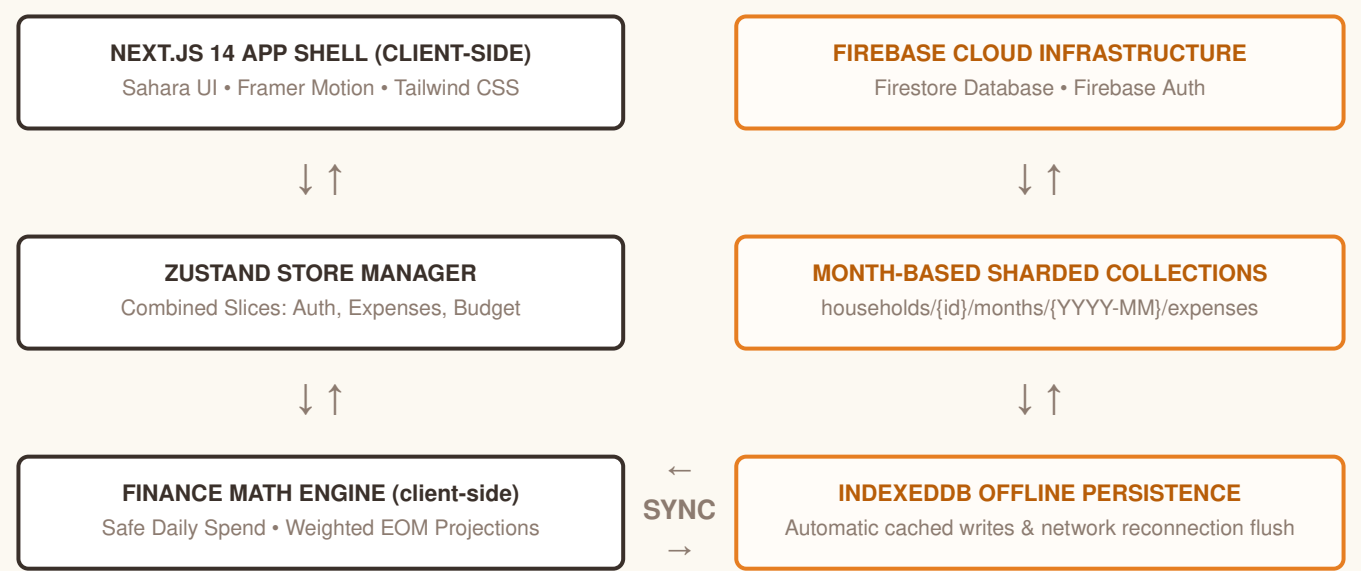
8. Feature Roadmap — What to Add vs. What to Remove

A perfect product requires cutting visual fluff to make room for high-utility student financial features. Below is the final product roadmap:

ACTION TYPE	FEATURE / UI MODULE	ENGINEERING & UX JUSTIFICATION
ADD	1-Tap "Quick Add" Widgets	College students make repeat daily purchases (e.g., ₹15 Chai, ₹20 Samosa, ₹50 Bus). Allow pinning 3 Quick-Add preset chips on the Dashboard to log an expense in a single tap without modal friction.
ADD	Daily Safe-Spend Odometer	Replace static "Remaining Balance" cards with the calculated S_{daily} allowance. Knowing "You can safely spend ₹245/day for the rest of August" is 10× more actionable for student cash-flow.
ADD	CSV / Excel Export Functionality	Students frequently need to report monthly expense ledgers to parents. Use client-side table export to generate clean CSV summaries directly from the loaded months array.
REMOVE	Inspirational Quote Cards	Stock wallet images and motivational quotes waste valuable viewport space on functional ledger pages. Restrict decorative cards to empty states (when a month has 0 transactions).
REMOVE	Staggered Text Animations	Character-by-character fade-in effects (<code>text-effect.tsx</code>) on primary headings feel sluggish after repeated use. Keep all UI page transitions under 150ms for snappy ergonomics.

9. System Architecture & Data Flow

Below is the complete structural architecture of SpendWise's Thick-Client setup, illustrating how local Zustand slices, mathematical engines, offline IndexedDB caches, and Firestore cloud shards interact:



— End of SpendWise Technical Audit & Remediation Document —